Linux szerver tananyag (6×45 perc)

1. foglalkozás – Csomagkezelés és frissítések

Elméleti bevezető (10 perc)

Debian alapú rendszereken a szoftverek csomagokba vannak szervezve. Az **APT** (Advanced Package Tool) kezeli a csomagok letöltését, telepítését, frissítését és törlését, valamint a függőségek kezelését. Az **`apt`** parancs olyan feladatokat végez, mint a rendszer csomaglistájának frissítése (`apt update`), csomagok telepítése (`apt install`), eltávolítása (`apt remove`), teljes törlése konfigurációval együtt (`apt purge`) és az elérhető frissítések telepítése (`apt upgrade`, `apt full-upgrade`)[1]. Az **`apt search`** vagy **`apt show`** csomaginformációt keres, az **`apt list --installed`** listázza a telepített csomagokat[2]. Az **`apt-get autoremove`** törli a már nem szükséges függőségeket, míg az **`apt-get clean`** kiüríti a letöltött csomagfájlok cache-ét[3].

Az **`dpkg`** a Debian csomagkezelés alacsony szintű eszköze. Csomagok kézi telepítésére (`dpkg -i`), eltávolítására (`dpkg --remove`), listázására (`dpkg -l`) vagy információ kiolvasására szolgál.

Gyakorlati rész (30–35 perc)

1. **Rendszerfrissítés és forráslista kezelése**

```
# csomaglista frissítése
sudo apt update

# elérhető frissítések telepítése (kérdés esetén válasszunk yes-t)
sudo apt upgrade

# felesleges függőségek eltávolítása
sudo apt autoremove

# csomag-gyorsítótár ürítése
sudo apt clean
```

Magyarázat: az `apt update` letölti az aktuális csomaglistát[1], az `apt upgrade` pedig az összes frissíthető csomagot telepíti. A `full-upgrade` (vagy régi néven `dist-upgrade`) kezeli a függőség-változásokat is.

1. **Csomagok keresése és telepítése**

Írjunk saját „parancsgyűjteményt” az alábbi szórakoztató csomagok telepítésével és kipróbálásával:

```
# telepítés előtt keressük meg a csomag nevét
apt search cowsay
apt show cowsay

# telepítsük a csomagokat
sudo apt install cowsay figlet fortune lolcat sl cmatrix neofetch htop tree
nmap nano

# próba: minden csomag futtatása
cowsay "Hello, Linux!"
figlet Debian
fortune | cowsay
sl     # vonat animáció
cmatrix # Mátrix hatás
neofetch
htop   # interaktív processz-monitor
tree   # könyvtárfa
nmap -V  # nmap verzió
nano --version
```

Az `htop` interaktívan jeleníti meg a folyamatokat, színes és egérrel is kezelhető[4]. A `tree` parancs telepítése után vizuálisan jeleníti meg a könyvtárstruktúrát[5].

1. **Csomagok eltávolítása és tisztítás**

```
# csomag eltávolítása (fájlkiszolgáló példa)
sudo apt remove cowsay

# teljes konfiguráció törlése
sudo apt purge figlet

# listázzuk a telepített csomagokat
apt list --installed | grep nano
```

1. **Mini-projekt: Saját parancsgyűjtemény**

Hozzunk létre egy `~/bin` mappát (ha nem létezik), majd hozzunk létre benne egy `parancsok.md` nevű jegyzetet nano segítségével. Minden telepített csomaghoz írjunk 1–2 mondatos leírást és példaparancsot. Használjuk a `nano` szerkesztőt:

```
mkdir -p ~/bin
nano ~/bin/parancsok.md
# a nano-ban: Ctrl+O ment, Ctrl+X kilép, Ctrl+W keresés[6]
```

Mentés után ellenőrizzük a fájl tartalmát:

```
cat ~/bin/parancsok.md
```

Ellenőrzés (5 perc)

- Sorold fel azokat a parancsokat, amelyekkel csomagot lehet telepíteni, frissíteni és eltávolítani!
- Mutasd meg, hogyan tudod listázni az összes telepített csomagot!
- Melyik parancs üríti a letöltött `.deb` fájlok gyorsítótárát[3]?

---

2. foglalkozás – Fájlrendszer és csatolások

Elméleti bevezető (10 perc)

A Linux fájlrendszert fastruktúrában szervezi – minden elérési út a gyökér `/` könyvtárból indul. A leggyakoribb fizikai fájlrendszer az **ext4**, amely a journaling (naplózó) támogatásnak köszönhetően megbízható és gyors[7]. A **`tmpfs`** egy memóriában tárolt fájlrendszer, amely gyors ideiglenes tárolást biztosít[8], míg a **`proc`** egy pseudo-fájlrendszer, amely a kernel adatszerkezeteit teszi elérhetővé fájlokként (pl. `/proc/cpuinfo`)[9].

A **mount** művelet egy eszközt (lemezpartíció, ISO, hálózati megosztás) kapcsol a könyvtárfához. A **`mount DEVICE DIRECTORY`** paranccsal csatoljuk az eszközt, `umount`-tal lecsatoljuk[10]. A konfigurációs fájl a `/etc/fstab`, amely soronként hat mezőt tartalmaz: **fájlrendszer**, **csatolási pont**, **típus**, **opciók**, **dump** és **pass**. A bejegyzések többek között `auto`, `noauto`, `ro` (csak olvasható), `rw` (írható), `exec` (végrehajtható) opciókat tartalmazhatnak[11]. UUID használata ajánlott, mivel független a lemez sorrendjétől[12].

A blokkeszközök listázására az **`lsblk`** parancsot használjuk; ez a `/sys` és az udev adatbázisból olvassa ki az eszközök adatait, és fastruktúrában jeleníti meg a meghajtókat[13]. A **`df`** parancs a fájlrendszerek foglaltságát (méret, foglalt, szabad) jeleníti meg[14], míg a **`du`** adott könyvtárak méretét számolja össze[15]. A **`blkid`** megjeleníti a partíciók fájlrendszer-típusát és UUID-jét[16].

Gyakorlati rész (30–35 perc)

1. **Fájl- és partícióinformációk gyűjtése**

```
# Lemezek fastruktúrája
lsblk

# Részletes információk egy eszközről (pl. sda)
sudo blkid /dev/sda

# Lemezek kihasználtsága
df -h        # humán olvasható
du -sh ~     # a saját home könyvtár mérete
```

1. **Virtuális lemez létrehozása és csatolása**

Mivel tantermi környezetben nem mindig áll rendelkezésre külön lemez, egy fájlalapú lemezt hozunk létre:

```
# 100 MB méretű üres fájl létrehozása
sudo dd if=/dev/zero of=/home/student/virtual_disk.img bs=1M count=100

# Loop eszköz társítása
sudo losetup -fP /home/student/virtual_disk.img
sudo losetup -a    # meglévő loop eszközök megtekintése

# ext4 fájlrendszer létrehozása
sudo mkfs.ext4 /dev/loop0

# Csatolási pont létrehozása
sudo mkdir -p /mnt/virtual_disk

# Csatolás
sudo mount /dev/loop0 /mnt/virtual_disk

# Ellenőrzés
df -h | grep loop0
```

Magyarázat: a `mount` hozzárendeli az eszközt a megadott könyvtárhoz; lecsatolni `sudo umount /mnt/virtual_disk` paranccsal lehet[10].

1. **`fstab` szerkesztése**

Nyissuk meg az `/etc/fstab` fájlt nano-val, és adjunk hozzá egy bejegyzést, hogy a virtuális lemez automatikusan csatolódjon induláskor UUID alapján:

```
sudo blkid /dev/loop0    # írd fel az UUID-t

sudo nano /etc/fstab
# fstab sor hozzáadása (példa):
# UUID=<kimásolt-uuid> /mnt/virtual_disk ext4 defaults 0 2
```

Mentsük el a fájlt, majd teszteljük a beállítást:

```
sudo umount /mnt/virtual_disk
sudo mount -a      # minden fstab bejegyzés csatolása
df -h | grep virtual_disk
```

1. **Home partíció azonosítása**

A saját home könyvtár partícióját a `df -h ~` és `lsblk` segítségével azonosíthatjuk. Jegyezzük fel a partíció nevét, fájlrendszer-típusát és méretét.

Ellenőrzés (5 perc)

- Mi a különbség az `ext4` és `tmpfs` fájlrendszerek között[7][8]?

- Sorold fel az `fstab` bejegyzés hat mezőjét[11]!
- Melyik parancs listázza a blokkeszközöket hierarchikusan[13]?

---

3. foglalkozás – Szöveg- és fájlkeresés, szűrés, feldolgozás

Elméleti bevezető (10 perc)

A rendszergazdai feladatokhoz elengedhetetlen a fájlok és szövegek gyors keresése és feldolgozása. Két alapvető parancs a **`find`** és a **`locate`**. A `find [útvonal] [kifejezés]` valós időben keresi meg a fájlokat különböző feltételek alapján (név, típus, méret, módosítási idő, jogosultság)[17]. Ezzel szemben a `locate` egy előre épített adatbázisból dolgozik; így gyors, de nem frissül azonnal. A `locate` opciói közé tartozik az `--ignore-case`, `--count` és `-e` (csak létező fájlok)[18].

Szövegszűrésre használható a **`grep`** (minták keresése szövegben)[19], a **`cut`** (oszlopok vagy karakterek kivágása)[20], a **`sort`** (sorok rendezése)[21], a **`uniq`** (duplikált sorok kiszűrése)[22] és a **`wc`** (sorok, szavak, karakterek számolása)[23]. A **`awk`** egy mintakereső és szövegfeldolgozó nyelv, amellyel sorokat, mezőket lehet kiválasztani és formázott kimutatásokat készíteni[24]. A **`sed`** stream-szerkesztő soronként transformálja a bemenetet; leggyakoribb használata a szöveg csere (`s/maszk/kulcs/`)[25].

Gyakorlati rész (30–35 perc)

1. **Fájlok keresése**

```
# Saját könyvtárban .txt kiterjesztésű fájlok keresése
find ~ -name "*.txt"

# Esetérzékeny vagy nagybetűt figyelmen kívül hagyó keresés
find /var/log -iname "*error*" -type f

# locate adatbázis frissítése és használata
sudo updatedb
locate passwd
locate -i bashrc | head -n 5    # kis- és nagybetű független
```

1. **Szövegkeresés logfájlokban**

```
# Rendszerüzenetek olvasása és szűrése
sudo dmesg | grep -i error | head

# /var/log/syslog fájlban a legutóbbi 100 sor vizsgálata
tail -n 100 /var/log/syslog | grep -E 'error|fail'
```

1. **Adatfeldolgozás csővezetékekkel**

```
# /etc/passwd felhasználónevek és shell-ek kivágása
cut -d: -f1,7 /etc/passwd | head
```

```
# Sorok rendezése és az ismétlődések megszámolása
cat /var/log/syslog | grep -o '[A-Z][a-z][a-z] [0-9]*' | sort | uniq -c |
sort -nr

# Szavak és karakterek számolása
wc -l /etc/passwd     # sorok
wc -w /etc/hosts      # szavak[26]
```

1. **awk és sed alapok**

```
# awk: harmadik mező kiírása /etc/passwd-ból
awk -F: '{print $1 " -> " $7}' /etc/passwd | head

# awk: fájlméretek összegzése
ls -l /var/log | awk '{sum += $5} END {print "Összesen:" , sum}'

# sed: minden 'test' szó kicserélése 'TESZT' -re
echo "This is a test file" | sed 's/test/TESZT/'
```

1. **Mini-projekt: logszűrő script**

Készíts egy `~/bin/logfilter.sh` nevű scriptet, amely paraméterként fogad egy logfájlt és egy kulcsszót, majd kiírja a kulcsszó előfordulását és darabszámát. A script használjon `grep`, `wc` és `awk` parancsokat. Futási jogosultság beállítása `chmod +x logfilter.sh`, majd teszteld a `/var/log/syslog` állományon.

Ellenőrzés (5 perc)

- Melyik parancs használ adatbázist a gyors kereséshez és milyen korlátja van[18]?
- Írd le a `grep -i` kapcsoló jelentését[19]!
- Hogyan tudod összeszámolni, hány sorból áll egy fájl[23]?

---

4. foglalkozás – Rendszeradminisztráció alapjai

Elméleti bevezető (10 perc)

Egy szerver üzemeltetésekor fontos megérteni, hogyan futnak a folyamatok és szolgáltatások. A **`ps`** parancs pillanatfelvételt készít az aktuális folyamatokról; a `ps aux` kimenetben szerepel a felhasználó, PID, %CPU, %MEM, VSZ, RSS, STAT és a parancs[27]. A **`top`** parancs valós időben frissíti ezt a listát, megmutatva a CPU-, memória- és swap használatot[4]. A **`htop`** egy fejlettebb, színes eszköz, amely egérrel is kezelhető; a funkcióbillentyűk (F1–F10) segítségével elérhető a help, beállítás, keresés, szűrés és folyamatok kilövése[28].

A folyamatok leállítására a **`kill`** és **`killall`** parancsok szolgálnak; alapértelmezés szerint `SIGTERM` jelet küldenek a PID-nek, de a `-9` opcióval a `SIGKILL` erőszakosabb megszakítás hajtható végre[29]. A **`pkill`** a folyamat nevét veszi paraméterként[30]. Prioritások

kezeléséhez használjuk a **`nice`** és **`renice`** parancsokat: a nagyobb nice-érték alacsonyabb prioritást jelent[31].

A háttérben és előtérben futó feladatok között a **`jobs`**, **`bg`**, **`fg`** parancsokkal válthatunk: a `jobs` listázza a futó vagy felfüggesztett feladatokat, a `bg` folytatja őket a háttérben, az `fg` az aktuális shell előterébe hozza vissza[32].

A szolgáltatásokat a **systemd** kezeli. A **`systemctl start/stop/status/restart`** utasításokkal indíthatjuk, állíthatjuk le, ellenőrizhetjük, illetve újraindíthatjuk a szolgáltatásokat[33]. A `enable/disable` parancs a rendszerindításkori automatikus indítást szabályozza[33]. A rendszer naplóit a **`journalctl`** parancs gyűjti; a `journalctl -n 10` utasítással a legutolsó 10 bejegyzést tekinthetjük meg[34]. A naplók idő, szolgáltatás vagy prioritás szerint szűrhetők[35].

Gyakorlati rész (30–35 perc)

1. **Folyamatok megfigyelése**

```
# Folyamatlista pillanatképe
ps aux | head

# Valós idejű megfigyelés
top

# htoppal interaktív nézet (kilépés: F10)
htop
```

Az `htop` felületén a funkcióbillentyűk: F1 (help), F2 (setup), F3 (search), F4 (filter), F5 (tree view), F6 (sort), F9 (kill)[28].

1. **Folyamatok leállítása és prioritások módosítása**

```
# Példa folyamat elindítása háttérben
sleep 120 &

# jobs lista
jobs

# háttérfolyamat leállítása
ps aux | grep sleep   # PID megkeresése
kill <PID>            # vagy killall sleep

# kill -9 erőszakos leállítás
kill -9 <PID>

# Prioritás csökkentése új folyamat indításakor
nice -n 10 yes > /dev/null &

# Futó folyamat prioritásának módosítása
sudo renice -n 5 -p <PID>
```

1. **Háttér/előterű feladatok kezelése**

```
# Parancs futtatása és felfüggesztése Ctrl+Z-vel
tail -f /var/log/syslog
# Ctrl+Z – felfüggesztés

# visszajuttatás háttérbe
bg

# előtérbe hozás
fg
```

1. **Szolgáltatások kezelése**

```
# SSH szolgáltatás státuszának ellenőrzése
sudo systemctl status ssh

# Indítás/leállítás/újraindítás
sudo systemctl start ssh
sudo systemctl stop ssh
sudo systemctl restart ssh

# Engedélyezés rendszerindításkor
sudo systemctl enable ssh
sudo systemctl disable ssh
```

1. **Naplók megtekintése**

```
# Legutóbbi 20 bejegyzés megjelenítése
journalctl -n 20

# Naplózás egy adott szolgáltatásra
journalctl --unit=ssh.service --since=today

# Magas prioritású hibaüzenetek
journalctl -p err -n 5
```

1. **Mini-projekt: rendszerfigyelő script**

Készíts egy `~/bin/sysmon.sh` nevű scriptet, amely 5 másodpercenként kiírja a 3 legnagyobb CPU-t használó folyamat nevét és azok CPU-használatát. Használd a `ps -eo pid,comm,%cpu --sort=-%cpu | head -n 4` parancsot, valamint `sleep 5`. Indítsd a scriptet háttérben, majd `kill` paranccsal állítsd le.

Ellenőrzés (5 perc)

- Melyik parancs ad folyamatosan frissülő listát a folyamatokról[4]?
- Hogyan tudsz a háttérben futó feladatot előtérbe hozni[36]?
- Mi a különbség a `kill` és a `kill -9` jelzés között[29]?

---

5. foglalkozás – Jogosultságok ismétlése és alkalmazása

Elméleti bevezető (10 perc)

A fájlokhoz és könyvtárakhoz három felhasználói osztály tartozik: **tulajdonos** (u), **csoport** (g) és **mindenki más** (o). Minden osztályhoz három alapjog van: olvasás (r=4), írás (w=2) és végrehajtás (x=1)[37]. A jogosultságok két módon állíthatók:

- **Szöveges (szimbolikus) mód** – az `chmod [ugoa][-+=]perms file` formátumban. A u, g, o, a jelöli a felhasználói osztályt; a +, -, = műveletek hozzáadást, elvételt vagy beállítást jelentenek[38]. Például `chmod g=r` csak olvasási jogot ad a csoportnak, `chmod a-x` mindenki számára megvonja a végrehajtási jogot[39].
- **Numerikus mód** – három vagy négy számjegy adja meg a tulajdonos, csoport és mások jogosultságát. Az értékek összege adja a jogokat: r=4, w=2, x=1; pl. `chmod 754 file` esetén a tulajdonos `rwx` (7), a csoport `r-x` (5), mások `r--` (4)[40]. A negyedik számjegy a setuid (4), setgid (2) és sticky (1) biteket jelöli[41].

A fájlok tulajdonosát a **`chown`** változtatja meg. Általános alakja: `chown [OPTIONS] USER[:GROUP] FILE`. Csak a felhasználó vagy a rendszergazda változtathatja meg a tulajdonost. A csoportot külön is módosíthatjuk a **`chgrp`** parancs segítségével: `chgrp [OPTIONS] GROUP FILE`[42]. A parancs `-R` opcióval rekurzívan is működik[43].

Az **`id`** parancs kiírja a jelenlegi vagy megadott felhasználó UID-jét, GID-jét és csoporttagságait[44]. Az `id -u` csak a felhasználó UID-jét, az `id -g` a GID-t adja vissza[45]. A **`groups`** parancs listázza a felhasználó elsődleges és kiegészítő csoportjait[46].

Gyakorlati rész (30–35 perc)

1. **Fájl és könyvtár jogosultságainak megtekintése**

```
# minta fájl létrehozása
touch ~/jogteszt.txt

# részletes listázás
ls -l ~/jogteszt.txt
# például: -rw-r--r-- 1 student student 0 okt 19 09:00 jogteszt.txt
```

1. **Jogosultságok módosítása szimbolikus módban**

```
# mindenki számára elveszünk futtatási jogot
chmod a-x ~/jogteszt.txt

# csak a csoport kap írási jogot
chmod g+w ~/jogteszt.txt

# tulajdonos és csoport ugyanazt a jogot kapja
chmod g+u ~/jogteszt.txt
```

1. **Jogosultságok módosítása numerikus módban**

```
# tulajdonos: rwx (7), csoport: r-x (5), mások: --- (0)
chmod 750 ~/jogteszt.txt

# sticky bit beállítása könyvtárra
mkdir ~/megosztott
chmod 1777 ~/megosztott
```

1. **Tulajdonos és csoport módosítása**

```
# új csoport létrehozása és felhasználó hozzáadása
sudo addgroup projektteam
sudo usermod -aG projektteam $USER

# csoport tagság ellenőrzése
id
groups

# fájl tulajdonosának módosítása
sudo chown $USER:projektteam ~/jogteszt.txt

# csoport módosítása chgrp-pel
sudo chgrp projektteam ~/jogteszt.txt
```

1. **Szimulált többfelhasználós megosztás**

Hozz létre egy `/srv/projektek` könyvtárat, állítsd be tulajdonosnak a `root:projektteam`-et, a jogosultságot 2775-re (setgid bittel). Ez biztosítja, hogy a csoport minden új fájl örökölje a csoport azonosítót. Próbálj létrehozni benne fájlokat saját és egy másik tanulói fiókkal, majd ellenőrizd a jogosultságokat.

1. **Mini-projekt: biztonságos mappastruktúra**

Tervezd meg és hozd létre a saját felhasználódon belül (pl. `~/szerver`) egy struktúrát, ahol:

- `public` mappába bárki olvashat és írhat (chmod 777),
- `team` mappát csak a `projektteam` tagok érhetik el (tulajdonos: te és projektteam csoport, jogosultság: 2770),
- `private` mappát csak te olvashatod (chmod 700).

Ellenőrizd a beállításokat `ls -l` segítségével.

Ellenőrzés (5 perc)

- Mi a különbség a szimbolikus és a numerikus `chmod` módszer között[38][40]?
- Milyen paraméterrel adhatunk sticky bitet egy könyvtárnak numerikus módban[41]?
- Hogyan listázod a saját csoporttagságaidat[47]?

---

6. foglalkozás – Összefoglaló projekt – kiszolgáló szimuláció

Elméleti bevezető (5 perc)

Az utolsó modulban egy „mini-szervert” hozunk létre, amely ötvözi az előző órák során tanultakat. Cél, hogy a diákok a saját virtuális gépükön belül egy többfelhasználós környezetet építsenek ki, ahol új felhasználók, csoportok, jogosultságok, csomagok, fájlrendszer-csatolások és folyamatkezelés is szerepel.

Gyakorlati rész (35–40 perc)

1. **Új felhasználók és csoportok létrehozása**

```
sudo addgroup admins
sudo addgroup students

sudo adduser alice
sudo adduser bob
sudo usermod -aG admins alice
sudo usermod -aG students bob

# ellenőrzés
id alice
id bob
```

1. **Közös mappák létrehozása**

```
sudo mkdir -p /srv/shared/projects
sudo chown root:admins /srv/shared
sudo chmod 2775 /srv/shared

sudo chown root:students /srv/shared/projects
sudo chmod 2770 /srv/shared/projects
```

1. **Csomagok telepítése**

Telepítsük a szolgáltatás-szimulációhoz szükséges csomagokat (pl. egy kis webszerver):

```
sudo apt update
sudo apt install apache2 nmap
sudo systemctl enable apache2
sudo systemctl start apache2
```

Ellenőrizzük a szolgáltatás állapotát:

```
sudo systemctl status apache2
sudo journalctl --unit=apache2 -n 5
```

1. **Fájlrendszer-bővítés és csatolás**

Ha van szabad virtuális lemez (vagy használjuk a korábbi `virtual_disk.img`-et), csatoljuk be `/srv/data` alá, és adjuk hozzá az `fstab`-hoz. A rendszer újraindításakor a csatolás automatikusan megtörténik.

1. **Folyamatkezelés**

Hozzunk létre egy folyamatosan futó folyamatot (pl. `watch -n 1 date`), majd figyeljük meg `htop`-pal, módosítsuk a nice értéket `renice` segítségével, és állítsuk le `kill`-lel.

1. **Önellenőrzés – feladatlista**

2. Hozz létre egy új felhasználót `carol` néven, add hozzá a `students` csoporthoz!

3. Készítsd el a `/srv/shared/projects/test.txt` fájlt úgy, hogy a `students` csoport írhatja, de más nem!
4. Állítsd be, hogy a `apache2` szolgáltatás induljon el automatikusan, és a logját nézd meg a `journalctl` segítségével!
5. Telepítsd a `cowsay` csomagot, és készíts egy üdvözlő üzenetet az új felhasználók számára!

Ellenőrzés (5 perc)

- Ellenőrizd, hogy az újonnan létrehozott felhasználók mely csoportok tagjai!
- Működik-e az automatikus csatolás az `fstab` bejegyzés alapján?
- Tudod-e listázni a futó folyamatokat és leállítani a tesztprogramot?

Visszatekintés és további tanulás

A hat modul során a diákok megtanulták a Debian alapú rendszerek kezelését csomagkezeléstől a jogosultságok beállításáig. A `find`, `grep`, `awk` és `sed` eszközökkel hatékonyan kereshetnek és szűrhetnek fájlokat; a `ps`, `top` és `htop` segítségével figyelhetik a futó folyamatokat; a `chmod`, `chown` és `chgrp` eszközökkel pedig biztonságos jogosultságmodellt alakíthatnak ki. Az összefoglaló projekt bizonyította, hogy a különálló modulokban tanult fogalmak integráltan is alkalmazhatók.

A tananyagban szereplő kódok minden Debian alapú rendszerben kipróbálhatók. A gyakorlás során javasolt jegyzetelni, a hibákat kijavítani, és kérdések esetén a `man` oldalakhoz vagy a hivatkozott forrásokhoz fordulni.

---

[1] [2] Chapter 8. The Debian package management tools

https://www.debian.org/doc/manuals/debian-faq/pkgtools.en.html

[3] APT Cheat Sheet | Packagecloud Blog

https://blog.packagecloud.io/apt-cheat-sheet/

[4] [27] [28] How to List All Processes in Linux {5 Different Commands}

https://phoenixnap.com/kb/list-all-processes-linux

[5] Print Linux Directory Structure as a Tree | Baeldung on Linux

https://www.baeldung.com/linux/print-tree-directory-structure

[6] How to Install and Use Nano Text Editor {+ Nano Commands}

https://phoenixnap.com/kb/use-nano-text-editor-commands-linux

[7] [8] [9] filesystems(5) - Linux manual page

https://man7.org/linux/man-pages/man5/filesystems.5.html

[10] How to Mount and Unmount File Systems in Linux | Linuxize

https://linuxize.com/post/how-to-mount-and-unmount-file-systems-in-linux/

[11] [12] fstab - Debian Wiki

https://wiki.debian.org/fstab

[13] How to List All Block Devices in Linux | lsblk Command - GeeksforGeeks

https://www.geeksforgeeks.org/linux-unix/lsblk-command-in-linux-with-examples/

[14] df command in Linux with Examples - GeeksforGeeks

https://www.geeksforgeeks.org/linux-unix/df-command-linux-examples/

[15] How to check disk space in Linux with df and du - SW Hosting's Blog

https://www.swhosting.com/en/blog/how-to-check-disk-space-in-linux-with-df-and-du

[16] blkid Command Examples in Linux - GeeksforGeeks

https://www.geeksforgeeks.org/linux-unix/blkid-command-examples-in-linux/

[17] How to Find a File in Linux | Find Command - GeeksforGeeks

https://www.geeksforgeeks.org/linux-unix/find-command-in-linux-with-examples/

[18] Linux locate Command: Syntax, Options, Examples

https://phoenixnap.com/kb/locate-command-in-linux

[19] grep Command in Linux With Examples

https://phoenixnap.com/kb/grep-command-linux-unix-examples

[20] cut command in Linux with examples - GeeksforGeeks

https://www.geeksforgeeks.org/linux-unix/cut-command-linux-examples/

[21] Linux sort Command with Examples

https://phoenixnap.com/kb/linux-sort

[22] uniq Command in Linux with Examples - GeeksforGeeks

https://www.geeksforgeeks.org/linux-unix/uniq-command-in-linux-with-examples/

[23] [26] Wc Command in Linux (Count Number of Lines, Words, and Characters) | Linuxize

https://linuxize.com/post/linux-wc-command/

[24] AWK Command in Linux with Examples {Statements, Patterns & Variables}

https://phoenixnap.com/kb/awk-command-in-linux

[25] Sed Command in Linux/Unix With Examples - GeeksforGeeks

https://www.geeksforgeeks.org/linux-unix/sed-command-in-linux-unix-with-examples/

[29] [30] How to Kill a Process in Linux from Command Line?

https://phoenixnap.com/kb/how-to-kill-a-process-in-linux

[31] Nice and Renice Command in Linux with Examples - GeeksforGeeks

https://www.geeksforgeeks.org/linux-unix/nice-and-renice-command-in-linux-with-examples/

[32] [36] Process Control Commands in Unix/Linux - GeeksforGeeks

https://www.geeksforgeeks.org/linux-unix/process-control-commands-unixlinux/

[33] [34] How to Start, Stop and Restart Services in Linux Using systemctl Command - GeeksforGeeks

https://www.geeksforgeeks.org/linux-unix/start-stop-restart-services-using-systemctl-in-linux/

[35] journalctl command in Linux

https://www.hostinger.com/tutorials/journalctl-command

[37] [38] [39] [40] [41] Chmod Command in Linux (File Permissions) | Linuxize

https://linuxize.com/post/chmod-command-in-linux/

[42] [43] Chgrp Command in Linux (Change Group) | Linuxize

https://linuxize.com/post/chgrp-command-in-linux/

[44] [45] id command in Linux with examples - GeeksforGeeks

https://www.geeksforgeeks.org/linux-unix/id-command-in-linux-with-examples/

[46] [47] Groups Command in Linux With Examples - GeeksforGeeks

https://www.geeksforgeeks.org/linux-unix/groups-command-in-linux-with-examples/